# CSE221: Algorithms

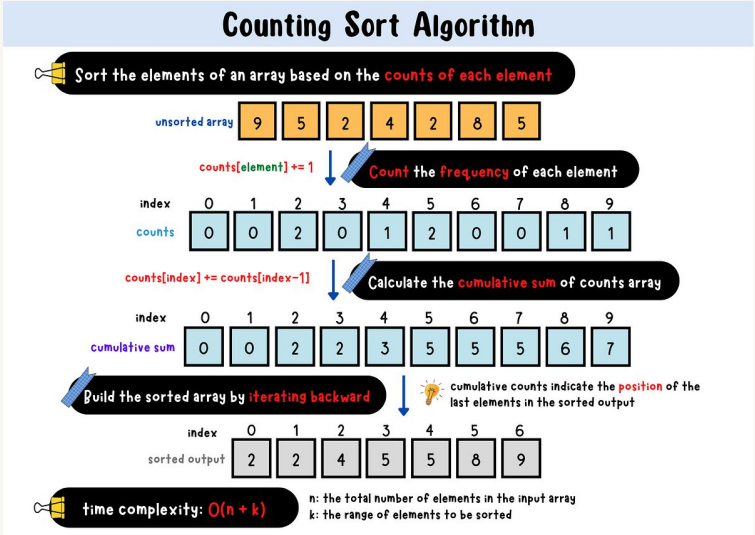
# Linear Time Sorting Algorithms

Prepared by: A B M Muntasir Rahman [MUNR]



# Counting Sort

- A sorting algorithm that sorts the elements of an array by counting the number of occurrences of each unique element in the array.
- **Counting Sort does not compare values** like the previous sorting algorithms we have looked at, and only works on non negative integers.



# Conditions

- **Non-negative integer values:** Counting Sort relies on counting the occurrences of distinct values, so the elements must be integers. With non-negative integers, each value can directly correspond to an index in the counting array. **If a value is negative, it cannot be mapped to a valid array index**, making the algorithm unsuitable without modification.

- **Limited range of values:** If the number of possible different values to be sorted is larger than the number of values to be sorted, the counting array we need for sorting will be larger than the original array we have that needs sorting, and **the algorithm becomes ineffective**.

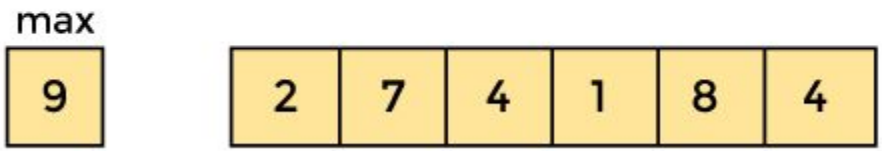
# Simulation

Let the elements of array are -



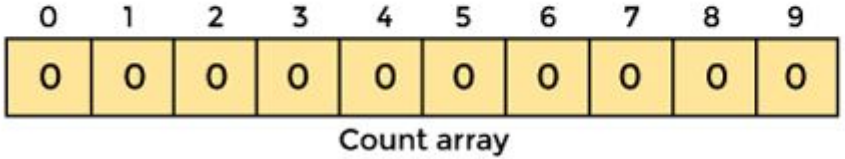
# Simulation

1. Find the maximum element from the given array. Let **max** be the maximum element.



# Simulation

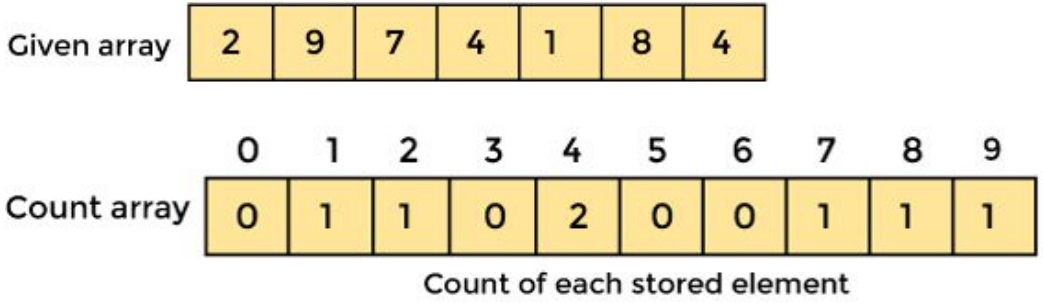
2. Now, initialize array of length **max + 1** having all 0 elements. This array will be used to store the count of the elements in the given array.



# Simulation

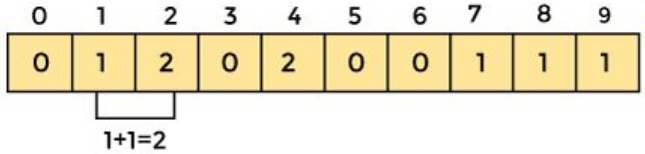
3. Now, we have to store the count of each array element at their corresponding index in the count array.

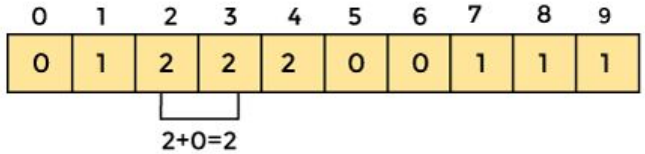
The count of an element will be stored as - Suppose array element '4' is appeared two times, so the count of element 4 is 2. Hence, 2 is stored at the $4^{th}$ position of the count array. If any element is not present in the array, place 0, i.e. suppose element '3' is not present in the array, so, 0 will be stored at $3^{rd}$ position.



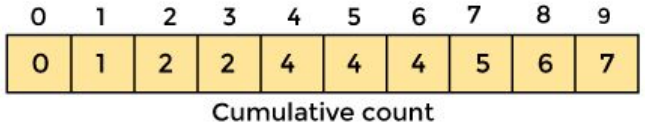
# Simulation

Now, store the cumulative sum of **count** array elements. It will help to place the elements at the correct index of the sorted array.



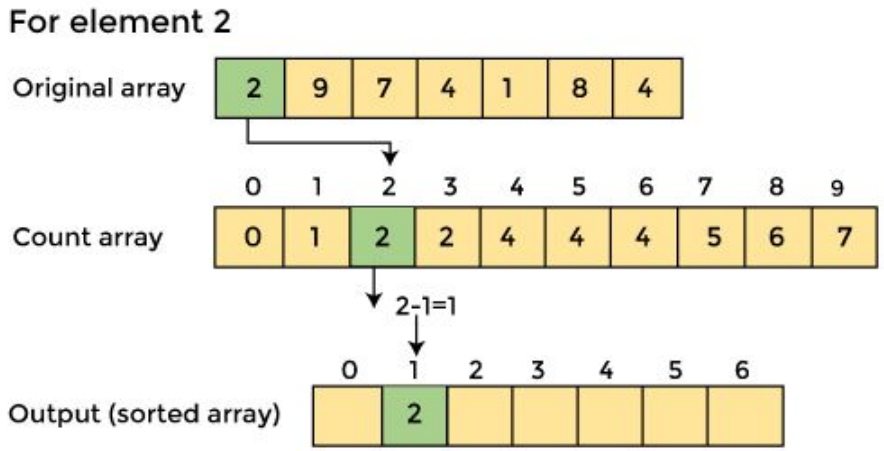




Similarly, the cumulative count of the count array is -

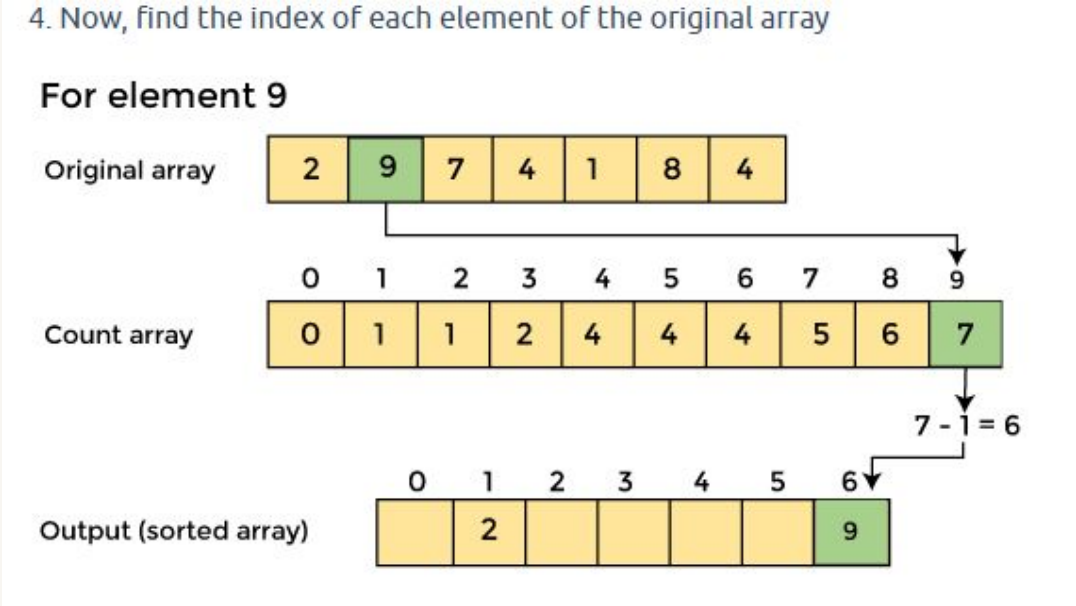


# Simulation

After placing element at its place, decrease its count by one. Before placing element 2, its count was 2, but after placing it at its correct position, the new count for element 2 is 1.

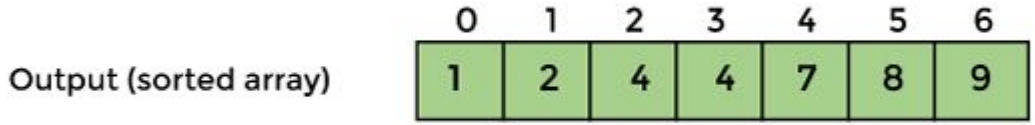


# Simulation


4. Now, find the index of each element of the original array
For element 9
Original array
2 9 7 4 1 8 4
0 1 2 3 4 5 6 7 8 9
Count array
0 1 1 2 4 4 4 5 6 7
7 - 1 = 6
0 1 2 3 4 5 6
Output (sorted array)
2 9

# Simulation

Similarly, after sorting, the array elements are -




Now, the array is completely sorted.

# Pseudocode

```
countingSort(array)
  max_val <- maximum value in array
  create count[0...max_val] initialized to 0

  for each num in array
    count[num] <- count[num] + 1

  for i <- 1 to max_val
    count[i] <- count[i] + count[i - 1]

  create sortedArray[0...sizeOfArray - 1]
  for each num in array (in reverse order)
     count[num] <- count[num] - 1
     sortedArray[count[num]] <- num

  return sortedArray
```

# Properties

- **Time Complexity:**
  - Best/Average/Worst case: O(n + k) where n is the number of elements in the input array, and k is the range of input values.
- **Space Complexity:**
  - Best/Average/Worst case: O(n + k) (for the count array and output array).
- **Stable:** Yes.
- **In-place:** No.
- Suitable for arrays where the range *(k)* of input values is not significantly larger than the number of elements *(n)*. Not suitable when k>>n, as it increases memory usage.